

计算机科学笔记示例

Violet

April 3, 2026

目 录

I	模板结构与排版	
1	模板概览	3
1	设计目标	3
1.1	标题层级示例	3
1.2	正文与强调	3
1.3	图片说明	4
2	列表、表格与公式	5
1	列表示例	5
2	表格示例	5
3	数学公式	5
II	组件与命令演示	
3	提示框与定义	8
1	提示框系统	8
2	定义与算法	8
4	代码与终端	10
1	代码块	10
2	快捷命令	10
3	终端与辅助命令	11
5	综合示例	12
1	一个小型知识片段	12
1.1	问题描述	12
1.2	结论归纳	12
1.3	补充说明	12
2	结束语	12

I

Part

模板结构与排版

1	模板概览	3
1	设计目标	3
2	列表、表格与公式	5
1	列表示例	5
2	表格示例	5
3	数学公式	5

Chapter 1

模板概览

1 设计目标

本示例用于演示 `computer-notes.cls` 的核心能力：标题层级、段落排版、列表样式、数学公式、提示框、代码块以及常用辅助命令。文档风格强调清晰、统一和可复用，适合作为后续笔记或课程讲义的起点。

这里原本适合放一张文档结构总览图，用于展示封面、目录、章节和组件之间的关系。当前用文字说明代替，后续你可以在对应位置补入总览截图。

1.1 标题层级示例

模板提供了七层标题体系，其中 `chapter`、`section`、`ssection`、`sssection`、`paragraph` 和 `subparagraph` 分别对应不同的视觉层级。实际写作时，建议把 `chapter` 用作主题分区，把 `section` 用作核心小节，再根据内容密度向下拆分。

五级标题

五级标题通常用于更细的知识拆分，适合在一节内部继续区分步骤、条件或注意事项。

六级标题

七级标题

六级和七级标题适合用于说明教程步骤中的细节，比如说在 SpringBoot 中集成 Nacos 配置中心时，分步详细说明每个步骤的操作细节和注意事项。

1.2 正文与强调

正文默认使用统一的中文字体和适中的行距。需要重点强调时，建议优先使用 **强调样式用于句子级别的表达**，而不是手动切换大量字体命令。对于需要视觉聚焦的专业术语首次出现时，可使用 **服务注册与发现**、**负载均衡** 这类高亮方式，后续重复出现时则恢复普通文本或使用加粗。

路径和文件名可以用 `src/main/java/App.java` 这样的方式标识，终端命令可用

```
mvn clean package
```

来显示。若只是想突出一个词，例如“**同步**”、“**异步**”或“**只读**”，更适合直接用加粗。

1.3 图片说明

如果某处暂时没有图片，可以先写成文字说明，例如“这里建议插入一张标题层级与排版效果的截图，用来展示蓝色标题盒、左侧竖线和正文缩进的整体观感”。这样不会让 LaTeX 把占位符当成非法语法处理。

Chapter 2

列表、表格与公式

1 列表示例

- 一级无序列表用于罗列并列概念。
 - 二级无序列表适合展开说明。
 - 子项 1
 - 子项 2
1. 有序列表适合表达步骤。
 2. 当步骤内部还需要分类时，可继续嵌套使用。
 - (a). 子步骤 A
 - (b). 子步骤 B

输入 需要处理的数据。

处理 对数据进行计算或转换。

输出 给出结果或结论。

2 表格示例

下面给出一个更贴近实际阅读的对比表，用于说明常见 List 实现的差异。这里改用 `tblr`，这样后续如果需要扩展表头、调整列宽或添加更复杂的对齐方式，会比传统 `tabular` 更灵活。

实现类	底层结构	访问特点	适用场景
ArrayList	动态数组	随机访问快	读多写少
LinkedList	双向链表	插入删除更灵活	频繁头尾操作
Vector	动态数组	同步方法较多	兼容旧代码

上面的表格可以继续扩展为更多列，例如“是否线程安全”“是否支持随机访问”“典型时间复杂度”等，这样更适合做知识总结。

3 数学公式

集合框架里经常会涉及复杂度分析。比如，哈希表平均查找复杂度接近 $O(1)$ ，红黑树查找复杂度通常为 $O(\log n)$ 。在实际笔记中，公式常用于说明“为什么这样选集合”，而不仅仅是展示符号本身。

$$T(n) = T\left(\frac{n}{2}\right) + O(1) \quad (3.1)$$

$$O(1) < O(\log n) < O(n) \quad (3.2)$$

$$O(n \log n) < O(n^2) \quad (3.3)$$

$$\text{负载因子} = \frac{\text{元素数量}}{\text{数组容量}}$$

II

Part

组件与命令演示

3	提示框与定义	8
1	提示框系统	8
2	定义与算法	8
4	代码与终端	10
1	代码块	10
2	快捷命令	10
3	终端与辅助命令	11
5	综合示例	12
1	一个小型知识片段	12
2	结束语	12

Chapter 3

提示框与定义

1 提示框系统

模板内置了七种提示框，适合在不同语义层面上表达信息。它们的设计目标不是“把内容包起来”，而是让读者一眼识别内容性质：是补充说明、实践建议、背景知识，还是风险提醒。

NOTE

这是一个笔记框。它适合放补充说明、记忆点或背景信息。

TIP

这是一个技巧框。适合放实现经验、优化建议或操作捷径。

INFO

这是一个信息框。适合放背景知识或通用说明。

WARNING

这是一个警告框。适合放可能出错、需要谨慎处理的内容。

CAUTION

这是一个注意框。适合强调容易忽略但影响较大的细节。

DANGER

这是一个危险框。适合放高风险操作、错误示范或明确禁止的行为。

TODO

这是一个待办框。适合标记后续还要补充的内容。

2 定义与算法

集合

由若干具有某种共同属性的对象组成的整体。这个定义框常用于放概念型内容，适合承接“是什么”而不是“怎么做”。

算法

解决问题的一组有限步骤，强调正确性、可执行性与效率。

Algorithm

排序问题的一个基本思路：

1. 从未排序区域中选择一个元素。
2. 将该元素插入到已排序区域的合适位置。
3. 重复直到所有元素都处理完成。

Chapter 4

代码与终端

1 代码块

代码块环境支持语法高亮、行号和文件标题。示例中优先展示常见语言，便于检查排版和着色是否正常。若后续你要写课程笔记，建议每个代码块都尽量配一句“这段代码在说明什么”。

ArrayListDemo.java

```
1 import java.util.ArrayList;
2 import java.util.List;
3
4 List<String> list = new ArrayList<>();
5 list.add("A");
6 list.add("B");
7 list.add(1, "X");
8
9 System.out.println(list);
```

fibonacci.py

```
1 def fibonacci(n):
2     if n <= 1:
3         return n
4     return fibonacci(n - 1) + fibonacci(n - 2)
5
6 print(fibonacci(10))
```

build.sh

```
1 #!/bin/bash
2 set -e
3
4 xelatex example.tex
5 xelatex example.tex
```

config.json

```
1 {
2   "name": "computer-notes",
3   "version": "1.0.0",
4   "enabled": true
5 }
```

2 快捷命令

短代码如果需要展示成完整结构，建议仍然使用独立代码块，因为这样更稳定，也更容易和正文区分开来。

`query.sql`

```
1 SELECT *  
2 FROM users  
3 WHERE id = 1;
```

3 终端与辅助命令

终端样式适合展示构建命令、Git 命令或系统命令。它的用途更偏向“操作指引”，而不是纯展示命令本身。

```
git status
```

```
xelatex example.tex
```

此外，行内命令可以这样写：`java -jar app.jar`。行内命令通常适合放在句子里，避免打断正文阅读节奏。

Chapter 5

综合示例

1 一个小型知识片段

下面用一个简短的主题把前面的组件串起来，模拟真实笔记中的写法。

1.1 问题描述

在 Java 中，集合框架的选择通常由四个维度决定：是否允许重复、是否需要顺序、是否需要随机访问、是否线程安全。若还要放到更接近工程实践的视角下看，通常还要考虑数据量大小、并发读写比例，以及后续是否需要排序或范围查询。

例如，在分布式系统里，一个注册中心常涉及 **服务注册与发现**；一个网关或客户端侧组件则经常涉及 **负载均衡**。这些场景都会影响你选择 Map、Set 还是队列结构。

1.2 结论归纳

- 如果强调随机访问，优先考虑 **ArrayList**。
- 如果强调去重，优先考虑 **HashSet**。
- 如果强调排序，优先考虑 **TreeMap** 或 **TreeSet**。
- 如果强调并发访问，优先考虑 **ConcurrentHashMap**。

1.3 补充说明

当需要强调某个关键结论时，可以写成：**遍历中修改普通集合，应优先使用迭代器自身的删除方法，因为这能避免快速失败带来的异常。**

示例结束前，建议检查字体、页眉、目录和代码块是否都符合预期。特别是标题层级、表格边框、提示框标题和代码块标题栏，这些位置最容易暴露模板样式问题。

这里原本可以放一张最终排版效果截图，用来对照封面、目录和正文区块之间的整体比例。当前先以文字说明替代，避免引入无效占位符。

2 结束语

这个示例文档的目标不是覆盖所有知识点，而是稳定展示模板的排版能力和常用命令。后续你可以直接基于这份结构复制出新的课程笔记文档。